

Quiz

Kruskal's algorithm on a connected graph $G = (V, E)$ has a runtime of $O(|E| + |V| \log(|V|))$.

True

☒ False

Kanten Sortieren $\geq \Omega(|E| \cdot \log(|E|))$

Suppose we run Kruskal's algorithm on a connected graph $G = (V, E)$. Then for a vertex $u \in V$, consider the quantity N_u defined as the number of times we change $\text{repr}[u]$. What is the tightest upper bound for N_u that holds for any connected graph?

a. $N_u \leq O(1)$

☒ b. $N_u \leq O(\log_2(|V|))$

c. $N_u \leq O(|V|)$

we always merge the smaller ZHK into the bigger

$\Rightarrow$ ZHK(u) must at least double every time we merge

$\Rightarrow$ can at most double $\log_2(|V|)$ times

Asymptotically, Floyd-Warshall's runtime is always at least as fast as Johnson's algorithm's runtime.

$O(n^3)$

$O(n \cdot (n+m) \log n)$

True

☒ False

$m \leq O(n)$

After the k -th iteration of the outer loop of the Floyd-Warshall algorithm, $d_{u,v}^k$ contains

a. the length of the shortest $u - v$ path using at most k edges

☒ b. the length of the shortest $u - v$ path that uses only intermediate vertices from $\{1, \dots, k\}$

definition of the subproblem

Suppose you run Floyd-Warshall on a directed, weighted graph $G = (V, E)$ with vertices $V = \{1, 2, 3\}$.

Assume the DP table of d^3 is as follows:

0	-2	-3
2	0	-1
3	1	0

Is it true that any such G will have no negative cycle?

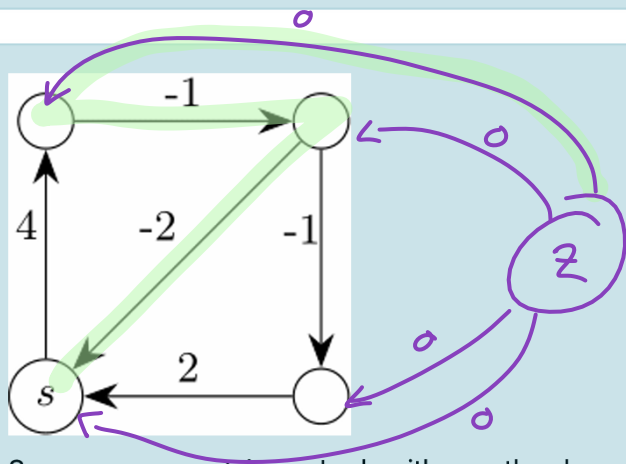
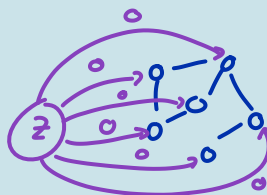
True
False

$$\exists \text{ neg cycle} \Leftrightarrow \exists v : d_{vv}^n < 0$$

Suppose we have a graph $G = (V, E)$ with weights $c : E \rightarrow \mathbb{R}$. Recall in Johnson's algorithm we use Bellman-Ford as a subroutine on a modified version of G to find a function $h : V \rightarrow \mathbb{R}$ such that we get a new set of weights $\hat{c} : E \rightarrow \mathbb{R}$ defined as $\hat{c}((u, v)) = c((u, v)) + h(u) - h(v)$ for $(u, v) \in E$ which guarantees $\hat{c}((u, v)) \geq 0$.

Then we have $h(v) \leq 0$ for all $v \in V$.

True
False



Suppose you run Johnson's algorithm on the above graph and compute the $h : V \rightarrow \mathbb{R}$ function as in lecture. What is the value of $h(s)$?

Answer:

-3

Let $G = (V, E)$ be a directed graph with vertices $V = \{1, 2, 3, 4\}$.

Let A_G be the *adjacency matrix* of G . Suppose that $(A_G)^3$ (i.e., the *adjacency matrix to the power 3*) is equal to

$$\begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

How many directed triangles (a collection of three edges which form a closed walk) does G have?

Answer:

1

a triangle has three nodes and is counted three times

Let $G = (V, E)$ be a directed graph, which has self loops on every vertex, and let A_G be its *adjacency matrix*.

Suppose there exists some $i \in \mathbb{N}$ such that A_G^i has all non-zero entries, then all pairs of vertices are reachable from one another.

True

False

Floyd - Warshall

Teilproblem: d_{uv}^i = Länge eines kürzesten Weges von u nach v , der nur Zwischenknoten aus $\{1, 2, \dots, i\}$ verwendet

Base case:
$$d_{uv}^0 = \begin{cases} 0 & \text{falls } u = v \\ c(u, v) & \text{falls } (u, v) \in E \\ \infty & \text{sonst} \end{cases}$$

Rekursion
$$d_{uv}^i = \min \left\{ \underbrace{d_{uv}^{i-1}}_{i \text{ kommt nicht vor}}, \underbrace{d_{ui}^{i-1} + d_{iv}^{i-1}}_{i \text{ kommt vor}} \right\}$$

Berechnungsreihenfolge:
 for $i = 1 \dots n$:
 for $u = 1 \dots n$:
 for $v = 1 \dots n$:
 compute d_{uv}^i

Lösung: d^n

Laufzeit: $O(n^3)$, Speicher $O(n^2)$

wichtig: $\exists$ negativer Zyklus $\Leftrightarrow \exists v \in V: d_{vv}^n < 0$

Johnson

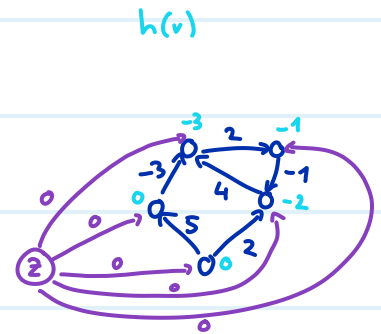
Ziel neue Kostenfunktion $\hat{c}(u, v)$ finden, die immer ≥ 0 ist, damit wir Dijkstra benutzen können

Höhenfunktion: $h(v)$ = Länge eines kürzesten Weges von z nach v

hat Eigenschaft $h(v) \leq h(u) + c(u, v)$

$$\Rightarrow c(u, v) + h(u) - h(v) \geq 0$$

wir definieren $\hat{c}(u, v) = c(u, v) + h(u) - h(v)$



Für einen Weg $W = (v_1, \dots, v_n)$ gilt:

$$\hat{c}(W) = c(v_1, v_2) + \cancel{h(v_1)} - \cancel{h(v_2)}$$

$$+ c(v_2, v_3) + \cancel{h(v_2)} - \cancel{h(v_3)}$$

$$+ c(v_3, v_4) + \cancel{h(v_3)} - \cancel{h(v_4)}$$

:

$$+ c(v_{n-1}, v_n) + \cancel{h(v_{n-1})} - h(v_n)$$

$$= c(W) + \underbrace{h(v_1) - h(v_n)}_{\text{unabhängig von welchem } v_1 - v_n \text{ Weg wir wählen}}$$

1) erstelle Knoten z und neue Kanten

$O(n)$

2) Berechne $h(v)$ mit Bellman-Ford

$O(nm)$

↳ abbrechen falls negativer Zyklus existiert

3) Berechne $\hat{c}(u, v)$

$O(m)$

4) n mal Dijkstra mit Gewichten $\hat{c}(u, v)$

$O(n(n+m) \log n)$

O-Notation Aufgaben

$$n^{\log n} \leq O(n^{100})$$

☐ True

☒ False

$$\lim_{n \rightarrow \infty} \frac{n^{\log n}}{n^{100}} = \lim_{n \rightarrow \infty} n^{\log n - 100} \quad \left. \vphantom{\lim_{n \rightarrow \infty}} \right\} \text{Exponent geht gegen } \infty$$

$$= \infty$$

$$\sqrt{n} \leq O((\log n)^{1000})$$

☐ True

☒ False

$$\lim_{n \rightarrow \infty} \frac{\sqrt{n}}{(\log n)^{1000}} \stackrel{\text{L'Hôp}}{=} \lim_{n \rightarrow \infty} \frac{\frac{1}{2\sqrt{n}}}{1000 (\log n)^{999} \cdot \frac{1}{n}} = \lim_{n \rightarrow \infty} \frac{\sqrt{n}}{2 \cdot 1000 \cdot (\log n)^{999}} \stackrel{\text{999 mal L'Hôp}}{=} \dots$$

$$= \lim_{n \rightarrow \infty} \frac{\sqrt{n}}{1000 \cdot 2^{1000}} = \infty$$

$$n^{1/\log_2 n} = \Theta(1)$$

☒ True

☐ False

$$n^{\frac{1}{\log_2 n}} = 2^{\log_2 \left(n^{\frac{1}{\log_2 n}} \right)} = 2^{\log_2(n) \cdot \frac{1}{\log_2(n)}} = 2^1 = 2$$

$\overset{\curvearrowright}{2^{\log_2}} \text{ Trick}$

$$2^{\sqrt{2 \log_2 n}} \leq O(n)$$

☒ True

☐ False

$$\sqrt{2x} = \sqrt{2} \cdot \sqrt{x} \leq x \quad \text{for large enough } x$$

$$\Rightarrow \text{also true for } x = \log_2 n: \quad \sqrt{2 \log_2 n} \leq \log_2 n$$

$$\Rightarrow 2^{\sqrt{2 \log_2 n}} \leq 2^{\log_2 n} = n$$

$$(\log_2 n)! \leq O(n)$$

☐ True

☒ False

$$\text{Substitution } n = 2^k: \quad (\log_2(2^k))! \leq O(2^k)$$

$$\Leftrightarrow k! \leq O(2^k) \quad \text{which is false}$$

DP Aufgaben

The Story: You are a master thief trying to flee from **City 1** to **City N**. Every city has a certain amount of loot you can steal, but some cities have heavy police presence. If you land in a city with a police station, you are immediately arrested.

You have two ways to travel from your current city i :

1. **Take a Taxi:** Move to city $i + 1$.
 - **Cost:** 1 Gold.
2. **Take the Express Train:** Move to city $i + 5$.
 - **Cost:** 5 Gold.
 - *Note: This allows you to "jump over" cities $i + 1$ through $i + 4$, avoiding any police stationed there.*

Goal: Calculate the maximum amount of gold you can have upon reaching City N . If it is impossible to reach City N (due to police blocking all paths), return -1 .

Input:

- N : Total number of cities.
- **Loot**: An array where $\text{Loot}[i]$ is the gold available in city i .
- **Police**: An array (or set) indicating if city i has a police station.

Teilproblem: $DP[i]$ = maximum gold the thief can have when he reaches city i , including the gold he can steal at city i .
We define it as $-\infty$ if he gets caught or runs out of money.

Dimension: $DP[1 \dots n]$

Base case: $DP[1] = \begin{cases} -\infty & \text{if } \text{police}[1] = \text{true} \\ \text{Loot}[1] & \text{else} \end{cases}$

Rekursion $DP[i] = \begin{cases} -\infty & \text{if } \text{police}[i] = \text{true} \\ \max \{ DP[i-1] - 1, DP[i-5] - 5 \} + \text{Loot}[i] & \text{else} \end{cases}$
 $\underbrace{\hspace{10em}}_{= -\infty \text{ if } < 0}$
 $\hspace{15em} = -\infty \text{ if } i \leq 5$

Berechnungsreihenfolge: for $i = 1 \dots n$

Lösung: $DP[n]$ or -1 if $DP[n] = -\infty$

Laufzeit: $O(n)$

DP in Code Expert (Left and Right, FS 22)

```
public static int getMaximumScore(int n, int[] A) {  
    int[][] DP = new int[n][n]; // maximum score from A[i] to A[j]  
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) DP[i][j] = -1;  
    return compute(0, n-1, DP, A);  
}  
  
public static int compute(int x, int y, int[][] DP, int[] A) {  
    if (DP[x][y] != -1) return DP[x][y]; // Memoization  
    if (x >= y) return 0; // Base Case  
  
    int cutBoth = compute(x+1, y-1, DP, A) + Math.abs(A[x]-A[y]);  
    int cutLeft = compute(x+1, y, DP, A);  
    int cutRight = compute(x, y-1, DP, A);  
    int max = Math.max(Math.max(cutLeft, cutRight), cutBoth);  
    DP[x][y] = max;  
    return max;  
}
```

Graph layering in Code Expert

```

public static int shortestGoodPath(int[][] G, int[] S, int k, int s, int t) {
    int n = G.length;
    DiGraph g = new DiGraph(n*(k+1)); // (k+1) Kopien der n Knoten
    // Idee: Layer l <=> l special nodes nacheinander benutzt

    for (int i = 0; i < n; i++) {
        for (int j : G[i]) {
            // wir iterieren über alle Kanten i-j
            if (S[j] == 1) { // j ist ein special node
                for (int l = 0; l < k; l++) {
                    g.addEdge(i+n*l, j+n*(l+1), 1); // wir gehen ein level nach oben
                }
            } else { // j ist ein normaler node
                for (int l = 0; l <= k; l++) {
                    g.addEdge(i+n*l, j, 1); // wir gehen zu level 0
                }
            }
        }
    }

    // check if starting s is special but no special nodes allowed
    if (S[s] == 1 && k == 0) return -1;

    if (S[s] == 1) {
        g.BFS(s+n); // start in layer 1 if s is special
    } else {
        g.BFS(s); // start in layer 0 if s is normal
    }

    int shortestWalk = 10000000;
    for (int l = 0; l <= k; l++) {
        shortestWalk = Math.min(shortestWalk, g.getDistanceTo(t+n*l));
    }
    if (shortestWalk < 1000000) return shortestWalk;
    return -1;
}

```

Lerntipps

- investiert zuerst ≤ 2 Tage pro Fach um die Theorie grob zu wiederholen
- danach so viele alte Prüfungen wie möglich lösen gerade ↓ ungerade ↑
- Programmier-Teile eventuell mehrmals lösen (A&D und Eprog)
- macht genug Sport
- normalerweise 8-10h lernen pro Tag, ~ 5 Tage Weihnachtsferien ist ok
- lernt lieber zu viel als zu wenig, im ersten Semester kann man sich schlecht einschätzen
- verschwendet eure Zeit nicht
- macht euch einen groben Lernplan
- Cheatsheets erst schreiben nachdem ihr ~ 4 Prüfungen gelöst habt

Prüfung

- analoge Uhr
- achtet auf Punkte pro Minute
- Ohropax
- Löst zuerst das was ihr am besten könnt